National Central University • Department of Computer Science & Information Engineering

CO3020 Computer Organization • Midterm Mock Examination [Student Examination Paper]

Student ID	_____	**Candidate Name**	_____
Exam Duration	100 Minutes (Closed-book, electronic devices strictly prohibited)	**Points / Problems**	100 Points / 8 Multi-part Problems (22 Sub-questions)

Exam Instructions:
1. This comprehensive examination synthesizes **Assignment 1 numerical calculations** and the **instructor-designated Chapter 3 core textbook problems** (Exercises 3.12, 3.13, 3.18, 3.27, 3.29, 3.35–3.37).
2. All calculation and design problems must include clear, step-by-step mathematical derivations and parameter substitutions. Numerical answers provided without intermediate derivation steps will receive zero credit.
3. Unless explicitly instructed otherwise, round all fractional results to two decimal places. Express machine instructions in standard 32-bit binary or hexadecimal notation (prefixed with `0x`).
4. Floating-point computations must strictly adhere to the IEEE 754 standard and the Round to Nearest Even mode.

Part I: Computer Abstractions, Performance & The Power Wall (Chapter 1) [Weight: 25 Points]

[10 Points] Question 1: Semiconductor Scaling, Moore's Law Doubling Time, and CMOS Dynamic Power Dissipation [Aligned with HW1 Q1]

Consider the desktop microprocessor process technology metrics from Intel between 2012 and 2020:

- **2012 (Ivy Bridge Architecture)**: Process feature size $t_1 = 22\ \text{nm}$, base clock frequency $f_1 = 3.4\ \text{GHz}$, supply voltage $V_1 = 1.20\ \text{V}$.
- **2020 (Tiger Lake Architecture)**: Process feature size $t_2 = 10\ \text{nm}$, base clock frequency $f_2 = 4.8\ \text{GHz}$, supply voltage $V_2 = 0.95\ \text{V}$.

Assume that the silicon die size remains constant across both microarchitectures, and that the total transistor density on the chip scales inversely with the square of the process feature size t: $\text{Density} \propto (1/t)^2$.

(a) (4 pts) Compute the transistor density scaling ratio from 2012 to 2020 (spanning 8 years), and determine the Compound Annual Growth Rate (CAGR) as a percentage (%).

SHOW FULL MATHEMATICAL DERIVATIONS AND FINAL VALUES HERE

(b) (3 pts) Based on the calculated CAGR, determine the doubling time (T_{double}, in years) required for the microprocessor's transistor count to double.

SHOW FULL MATHEMATICAL DERIVATIONS AND FINAL VALUES HERE

(c) (3 pts) The CMOS dynamic switching power formula is defined as $P = C \times V^2 \times f$. Assume that due to gate insulator improvements, the effective capacitive load C in 2020 is reduced by 20% compared to 2012 ($C_{2020} = 0.80 \times C_{2012}$). Compute the dynamic power ratio P_{2020}/P_{2012}, and state explicitly whether the dynamic power increased or decreased.

SHOW FULL MATHEMATICAL DERIVATIONS AND FINAL VALUES HERE

[15 Points] Question 2: CPU Performance Equation, Multi-Class Weighted Average CPI, Compiler Optimization, and Amdahl's Law Physical Limit [Aligned with HW1 Q2, Q3, Q4]

A benchmark application has a dynamic instruction count of $IC = 2.0 \times 10^9$ instructions. The instruction mix breakdown and per-class CPI for two candidate processor implementations are given in the table below:

Instruction Class	Proportion of Total IC (%)	Processor P1 CPI (Clock: 2.5 GHz)	Processor P2 CPI (Clock: 3.2 GHz)
Class A (ALU Operations)	40%	1	2
Class B (Conditional Branches)	20%	2	2
Class C (Load / Store)	30%	4	3
Class D (Floating-Point)	10%	5	4

(a) (4 pts) Calculate the global weighted average CPI for both Processor P1 and Processor P2. Based on total CPU execution time, determine which processor executes the benchmark faster and compute the performance speedup factor.

SHOW FULL MATHEMATICAL DERIVATIONS AND FINAL VALUES HERE

(b) (5 pts) Compiler Optimization Impact: A newly engineered optimizing compiler, Compiler Y, restructures loop memory scheduling and successfully reduces the number of Class C (Load/Store) instructions by 50%, while leaving the dynamic counts of Classes A, B, and D unaffected. When executing on Processor P1, calculate: (i) the new total dynamic instruction count, (ii) the new weighted average CPI, (iii) the new execution time, and (iv) the speedup relative to the baseline compiler.

SHOW FULL MATHEMATICAL DERIVATIONS AND FINAL VALUES HERE

(c) (6 pts) Amdahl's Law and Single-Component Physical Limit: Consider the baseline program running on Processor P1 (baseline execution time $T_0 = 2.0$ seconds). Engineering management proposes: "Focus microarchitectural redesign exclusively on optimizing the branch execution unit (Class B) to reduce total program execution time by 25%."

(i) Calculate the total execution time spent exclusively on branch instructions in the baseline program.

(ii) If branch execution time is reduced to its theoretical physical limit (0 seconds), what is the maximum possible execution time reduction achievable for the overall application?

(iii) Using rigorous mathematical derivation based on Amdahl's Law, determine whether this 25% execution time reduction target is physically attainable. Clearly state your verdict (Attainable or Physically Unattainable) and justify.

SHOW FULL MATHEMATICAL DERIVATIONS AND FINAL VALUES HERE

Part II: Instructions: Language of the Computer (Chapter 2) [Weight: 25 Points]

[10 Points] Question 3: MIPS Instruction Format Deconstruction, Field Binary Expansion, and 32-Bit Hexadecimal Machine Code Assembly [Aligned with HW1 Q6]

Examine the hardware mapping between MIPS assembly instructions and 32-bit hardware register fields.

(a) (5 pts) R-type Instruction Encoding: Assemble the MIPS assembly instruction `add $s2, $t0, $t1` into 32-bit machine code.
Given register numbers: `$t0 = $8, $t1 = $9, $s2 = $18`, and arithmetic function code `funct = 32`.
(i) Identify the 6 instruction fields and their decimal values in hardware encoding order (note the register operand layout!): `op, rs, rt, rd, shamt, funct`.
(ii) Expand each field into its binary bit sequence.
(iii) Output the final 32-bit machine instruction as an 8-digit hexadecimal value (e.g., `0x01099020`).

SHOW FIELD DECOMPOSITION, BINARY STRING, AND HEXADECIMAL CODE HERE

(b) (5 pts) I-type Instruction Disassembly and Branch Target Addressing: Given the hardware instruction fields below:
• `op = 0x23` (hexadecimal)
• `rs = 29` (stack pointer `$sp`)
• `rt = 31` (return address `$ra`)
• `immediate = 8` (decimal)
(i) Write the corresponding standard MIPS assembly instruction syntax.
(ii) Write its 32-bit hexadecimal machine code.
(iii) Extension: Suppose a conditional branch instruction `beq $s0, $s1, 16` (where `immediate = 16`) is located at byte memory address `0x00400030`. Calculate the exact hexadecimal target memory address (Target PC) taken when the branch evaluates to true.

SHOW ASSEMBLY SYNTAX, HEXADECIMAL CODE, AND PC-RELATIVE CALCULATION HERE

[15 Points] Question 4: Loop Reverse Engineering, Closed-Form Dynamic Instruction Count, and Strided 2D Array Addressing [Aligned with HW1 Q7, Q9]

Analyze the following MIPS assembly loop segment (assume delayed branching is disabled and integer arithmetic does not overflow):

```
# Initial state: t1 contains non-negative integer N (N >= 0), s2 is initialized to 0
LOOP:
    slt  t2, 0, $t1       # Step 1: Evaluate 0 < $t1
    beq  t2, 0, DONE      # Step 2: Exit loop if condition is false
    addi t1, t1, -1       # Step 3: Decrement loop counter by 1
    addi s2, s2, 3        # Step 4: Accumulate sum by adding 3
    j    LOOP             # Step 5: Unconditional jump back to LOOP
DONE:
```

(a) (7 pts) Loop Reverse Engineering and Dynamic Instruction Counting:
(i) (2 pts) Reverse engineer the assembly loop into equivalent high-level C code (use variable `i` for `$t1` and `B` for `$s2`).
(ii) (2 pts) If register `$t1` is initialized to $N = 12$, what is the final value stored in `$s2` upon reaching `DONE`?
(iii) (3 pts) Derive a closed-form mathematical formula for the total dynamic basic MIPS instructions executed from entering `LOOP` until final exit to `DONE` for any initial integer $N \geq 0$ (including the final termination evaluation). Calculate the exact dynamic instruction count for $N = 12$.

SHOW C CODE SNIPPET, FINAL ACCUMULATOR VALUE, AND INSTRUCTION COUNT FORMULA HERE

(b) (8 pts) Strided Array Access `D[4*j]` Byte Offset and Assembly Implementation:
Consider the following nested C loop:

```
for (i = 0; i < a; i++) {
    for (j = 0; j < b; j++) {
        D[4 * j] = i + j;
    }
}
```

Assume integer variables `a, b, i, j` reside in registers `$s0, $s1, $t0, $t1`. The base memory address of array `D` resides in `$s2`. Each array element is a 32-bit signed integer (4 bytes), word-aligned, and does not cause overflow.
(i) (3 pts) Rigorously derive the formula for the **memory byte offset** of element `D[4*j]` relative to base address `$s2`. If logical shift left (`sll`) is utilized to compute this multiplication, what shift amount (`shamt`) must be configured?
(ii) (5 pts) Write complete, highly optimized MIPS assembly code implementing this nested loop using the minimum number of instructions. Maintain the initial values of `$s0, $s1, $s2` intact.

SHOW BYTE OFFSET DERIVATION AND OPTIMIZED MIPS ASSEMBLY CODE HERE

Part III: Computer Arithmetic & Floating-Point Systems (Chapter 3) [Weight: 50 Points]

[15 Points] Question 5: Baseline Multiplier vs. Optimized Multiplier 6-Cycle Hardware Trace & Verification [Aligned with Exercises 3.12, 3.13]

Compute the multiplication of two 6-bit unsigned integers:

- **Multiplicand (M)**: Octal $45_8 = 37_{10} = 100101_2$.
- **Multiplier (Q)**: Octal $13_8 = 11_{10} = 001011_2$.

Expected mathematical product: $37 \times 11 = 407_{10} = 0001\ 1001\ 0111_2 = 627_8$.

(a) (8 pts) Baseline Multiplier Hardware Trace (Textbook Figure 3.3 Architecture):
Hardware register setup: Multiplicand is placed in a **12-bit register** (initial value in the lower 6 bits: `0000 0010 0101`, shifted left by 1 bit each cycle); Multiplier is placed in a **6-bit register** (initial value: `001011`, shifted right by 1 bit each cycle); Product is placed in a **12-bit register** (initial value: `0000 0000 0000`).
Provide the complete 6-cycle execution trace table from Iteration 0 through Iteration 6.

DRAW OR FILL IN THE CYCLE-BY-CYCLE HARDWARE TRACE TABLE FOR ITERATION 0 THROUGH 6

(b) (7 pts) Optimized Multiplier Hardware Trace (Textbook Figure 3.5 Architecture):
The multiplicand is held stationary in a **6-bit register** ($M = 100101_2$); Product and Multiplier share a single **12-bit register [A | Q]**, where the upper 6 bits A are initialized to `000000`, and the lower 6 bits Q hold the multiplier `001011`.
In each iteration: if $Q_0 = 1$, compute $A = A + M$, then shift the **entire 12-bit register [A | Q] right by 1 bit**.
Record the value of [A | Q] after addition and after shifting for each of the 6 cycles, and verify the final 12-bit output.

FILL IN THE 6-CYCLE RIGHT-SHIFT TRACE TABLE FOR [A | Q]

[10 Points] Question 6: Restoring Division 6-Cycle Hardware Trace & Signed Division Sign Rules [Aligned with Exercise 3.18 & Slides]

Compute the division of two 6-bit unsigned integers:

- **Dividend (D)**: Octal $62_8 = 50_{10} = 110010_2$.
- **Divisor (d)**: Octal $12_8 = 10_{10} = 001010_2$.

(a) (7 pts) Restoring Division Step-by-Step Hardware Trace (Textbook Figure 3.8 / 3.10 Architecture):
Divisor is fixed in a 6-bit register (`001010`); Remainder and Quotient share a **12-bit register [Remainder | Quotient]**, initialized with the upper 6 bits set to `000000` and lower 6 bits holding the dividend `110010`.
Per-cycle sequence: (1) Shift [Remainder | Quotient] left by 1 bit; (2) Remainder = Remainder - Divisor; (3) Evaluation: If Remainder < 0, restore remainder (add Divisor back) and set quotient LSB $q_0 = 0$; if Remainder ≥ 0, keep remainder and set quotient LSB $q_0 = 1$.
Complete the full 6-cycle trace table, and state the final Quotient and Remainder.

FILL IN THE 6-CYCLE RESTORING DIVISION TRACE TABLE WITH SHIFT LEFT, SUBTRACT, AND RESTORE STEPS

(b) (3 pts) Signed Division Sign Rules: Write the fundamental arithmetic identity connecting Dividend (D), Divisor (d), Quotient (q), and Remainder (r). Compute the quotient and remainder for:
(i) $(+23) \div (-5)$ (ii) $(-23) \div (+5)$ (iii) $(-23) \div (-5)$
Explain why computer hardware strictly enforces the rule that "the sign of the remainder must agree with the sign of the dividend."

STATE ARITHMETIC IDENTITY, THREE QUOTIENT/REMAINDER PAIRS, AND ARCHITECTURAL RATIONALE

[15 Points] Question 7: 16-Bit Half-Precision Range, Precision, and GRS Rounded Floating-Point Addition [Aligned with Exercises 3.27, 3.29]

Consider the IEEE 754 16-bit Half-Precision floating-point standard:

- **Sign bit (S)**: 1 bit
- **Biased Exponent (E)**: 5 bits, with bias value $\text{Bias} = 15$ (Excess-15 system, unbiased exponent $e = E - 15$)
- **Fraction (F)**: 10 bits, normalized significand has an implicit leading 1 ($1.F$)

(a) (5 pts) Dynamic Range and Precision Boundaries (Exercise 3.27 Variant):
(i) (2 pts) Provide the 16-bit binary pattern and decimal value for both the **smallest positive normalized number** and the **largest positive normalized number**.
(ii) (1 pt) Determine the machine epsilon (relative precision unit, ϵ_{mach}) of this 16-bit format.
(iii) (2 pts) If the system designer alters the exponent bias to $\text{Bias} = 16$ (Excess-16), analyze the specific changes to the dynamic range and the underflow threshold relative to the standard Excess-15 system.

SHOW EXTREME VALUES, MACHINE EPSILON, AND EXCESS-16 COMPARISON ANALYSIS HERE

(b) (10 pts) Floating-Point Addition Four Steps and GRS Rounding (Exercise 3.29 Variant):
Perform manual floating-point addition in this 16-bit half-precision format for:
• $A = 26.125_{10}$
• $B = 0.4150390625_{10}$
Demonstrate the full four-step arithmetic derivation:
(i) (2 pts) Convert values A and B into binary scientific notation and specify their 16-bit representations (S, E, F).
(ii) (2 pts) Align binary points: shift the significand of the number with the smaller exponent to the right. State the shift distance and write the aligned significand.
(iii) (2 pts) Extract the exact values (0 or 1) of the **Guard bit (G)**, **Round bit (R)**, and **Sticky bit (S)** generated by the alignment shift.
(iv) (2 pts) Perform significand addition and determine whether post-normalization is required.
(v) (2 pts) Apply the IEEE 754 **Round to Nearest Even** rule to determine whether rounding up occurs. State the final 16-bit binary pattern and its 4-digit hexadecimal machine code representation.

SHOW FOUR-STEP FLOATING-POINT ADDITION, GRS BIT EXTRACTION, ROUNDING EVALUATION, AND FINAL HEXADECIMAL CODE HERE

[10 Points] Question 8: IEEE 754 Gradual Underflow, Precision Limits on Integers, and Breakdown of the Associative Law [Aligned with Exercises 3.35, 3.36, 3.37]

Investigate representation limits and computer algebraic properties of the IEEE 754 32-bit Single-Precision format (1 sign bit, 8 exponent bits, 23 fraction bits).

(a) (3 pts) Subnormal Numbers and Underflow Mechanism (Exercise 3.35 Variant):
(i) State the decimal value of the smallest positive normalized number in single precision.
(ii) Explain the architectural rationale for introducing subnormal numbers ($0.F \times 2^{-126}$ with an exponent field of all zeros) to achieve gradual underflow.
(iii) Compute the absolute smallest positive non-zero subnormal number representable in this format.

SHOW SMALLEST NORMALIZED VALUE, GRADUAL UNDERFLOW RATIONALE, AND SMALLEST SUBNORMAL VALUE HERE

(b) (3 pts) Upper Limit of Consecutive Exact Integers (Exercise 3.36 Variant):
(i) Identify the smallest positive integer N that cannot be represented exactly in IEEE 754 single precision.
(ii) Provide a rigorous mathematical proof based on the 24-bit significand precision (23 fraction bits + 1 implicit leading 1).

STATE INTEGER N AND PROVIDE MATHEMATICAL PROOF BASED ON SIGNIFICAND PRECISION HERE

(c) (4 pts) Breakdown of the Associative Law (Exercise 3.37 Variant):
Given three single-precision floating-point variables: $A = 1.0 \times 10^{30}$, $B = -1.0 \times 10^{30}$, and $C = 1.0$.
(i) Compute the result of $(A + B) + C$ in floating-point hardware.
(ii) Compute the result of $A + (B + C)$ in floating-point hardware.
(iii) Explain why the two evaluations yield completely different results. Why do modern optimizing compilers by default prohibit reordering floating-point operations according to algebraic associativity?

SHOW STEP-BY-STEP EVALUATIONS, EXPLAIN PRECISION ABSORPTION, AND DISCUSS COMPILER OPTIMIZATION CONSTRAINTS